Dictionaries and Tuples

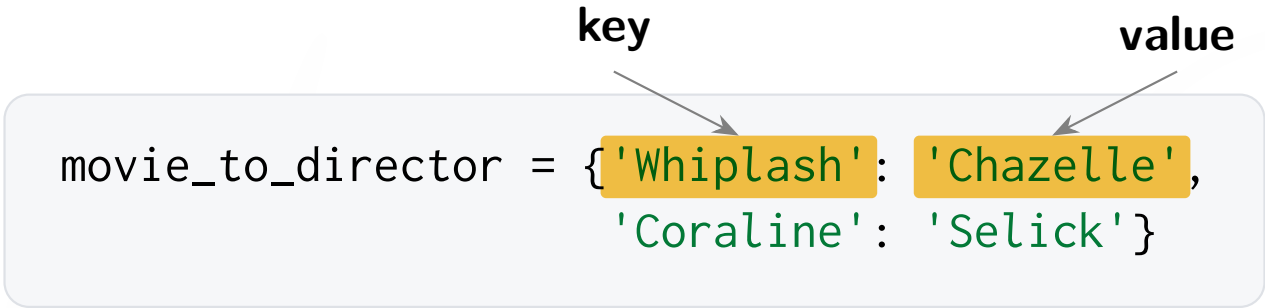
Introduction to Computer Programming

Chandra Gummaluru

chandra-gummaluru.github.io



A **dictionary** (`dict`) stores **key/value pairs**. We look a value up by its key, not by a position.



its type is `dict[str, str]`: keys first, then values

Square brackets with a key look up its value. A key that isn't in the `dict` raises a `KeyError`.

```
movie_to_director = {'Whiplash': 'Chazelle',
                     'Coraline': 'Selick'}
```

```
>>> movie_to_director['Whiplash']
'Chazelle'
>>> movie_to_director['Shrek']
KeyError: 'Shrek'
```

`in` checks the keys, not the values. `len` gives the number of key/value pairs.

```
movie_to_director = {'Whiplash': 'Chazelle',
                     'Coraline': 'Selick'}
```

```
>>> movie_to_director['Whiplash']
'Chazelle'
>>> movie_to_director['Shrek']
KeyError: 'Shrek'
>>> 'Coraline' in movie_to_director
True
>>> 'Selick' in movie_to_director
False
>>> len(movie_to_director)
2
```

dict_demo.py

Program

```
# Try these in the shell below:
#   animal_to_sound = {'cow': 'moo', 'owl': 'hoot'}
#   animal_to_sound['owl']
#   'moo' in animal_to_sound
#   animal_to_sound['cat'] = 'meow'
#   animal_to_sound
```

33 responses

Which of the expressions (a) to (e) evaluate to `True`?

(a) `'dog' in name_to_binomial`

(b) `'Canis familiaris' in name_to_binomial`

(c) `name_to_binomial[0] == 'human'`

(d) `len(name_to_binomial) == 3`

```
   name_to_binomial == {'dog': 'Canis familiaris', 'narwhal': 'Monodon monoceros',
'human': 'Homo sapiens'}
```

```
1  name_to_binomial = {'human': 'Homo sapiens',
2                      'dog': 'Canis familiaris',
3                      'narwhal': 'Monodon monoceros'}
```

✓	(a), (d) and (e)	**27%**	9 votes
B	(a) and (d) only	**9%**	3 votes
C	(a), (b), (d) and (e)	**6%**	2 votes
D	(a), (c), (d) and (e)	**58%**	19 votes

Each key appears in a `dict` only once, and keys must be **immutable**. Values can be anything, including `lists`.

immutable	**mutable**
can be keys	can't be keys
`int` `float` `bool`	`list` `dict`
`str` `tuple`	

A `dict` is mutable: we can add, change and remove its pairs.

A **tuple** is an immutable sequence, written with round brackets. It works like a `list` except that it can't change, so it can be a `dict` key.

```
>>> point = (3, 4)
>>> point[0]
3
>>> point + (5,)
(3, 4, 5)
>>> point[0] = 7
TypeError
```

```
>>> one = ('cat',)
>>> len(one)
1
>>> len(('cat'))
3
>>> stops = {(3, 4): 'Bloor'}
```

A `tuple` with one item needs a comma: `('cat')` is just the string `'cat'`. In this course, a `tuple` of strings has type `tuple[str]`.

Assigning to a key adds a new pair, or replaces the value if the key is already there.

```
>>> movie_to_rating = {'Shrek': 8}
>>> movie_to_rating['Coraline'] = 8
>>> movie_to_rating['Shrek'] = 9
>>> movie_to_rating
{'Shrek': 9, 'Coraline': 8}
```

adds a pair

replaces the value

A value that is a `list` can be changed with `list` methods.

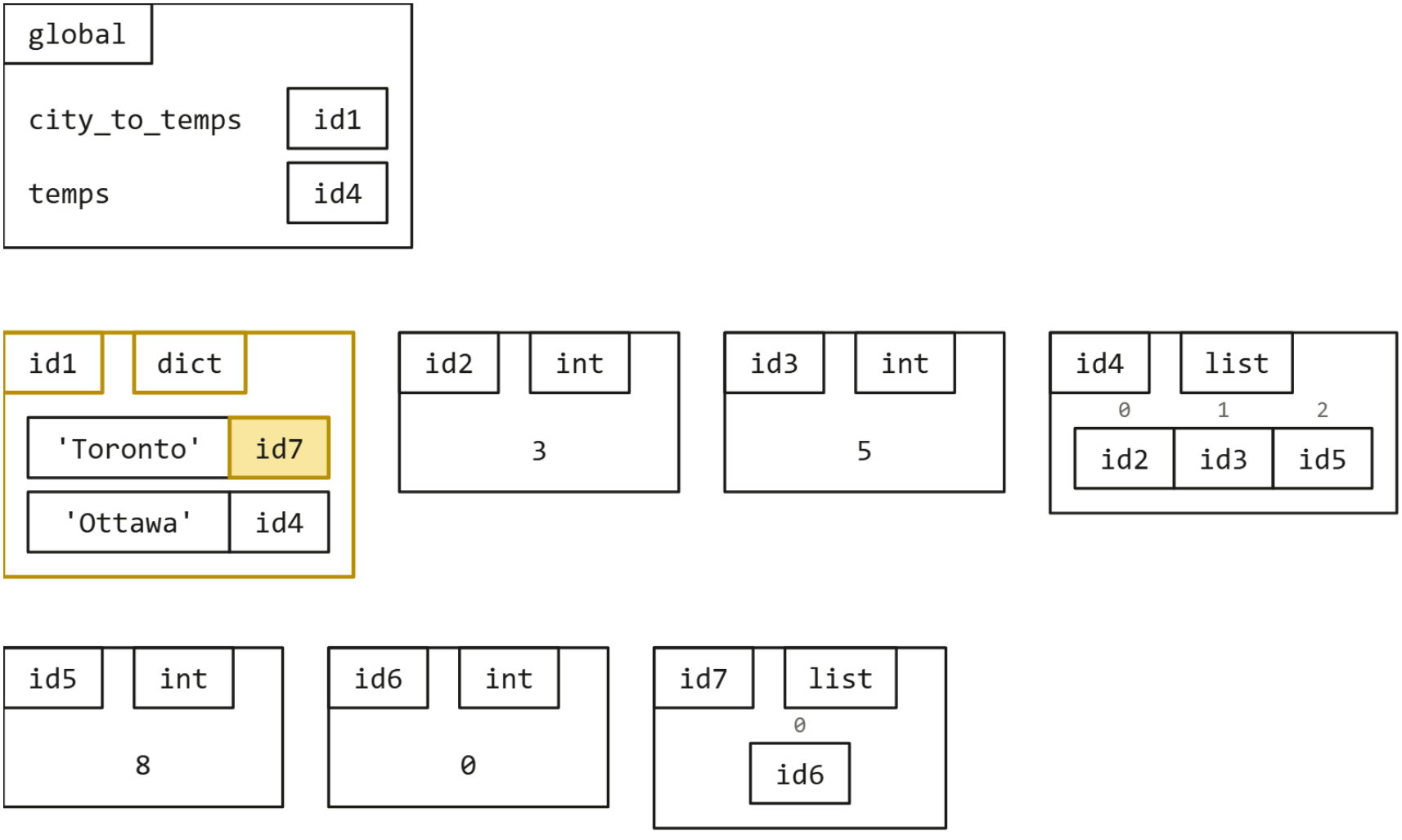
```
>>> movie_to_actors = {'Shrek': ['Myers']}
>>> movie_to_actors['Shrek'].append('Murphy')
>>> movie_to_actors
{'Shrek': ['Myers', 'Murphy']}
```

dict_memory.py

Program

```
1  city_to_temps = {'Toronto': [3, 5]}
2  temps = city_to_temps['Toronto']
3  temps.append(8)
4  city_to_temps['Ottawa'] = temps
5  city_to_temps['Toronto'] = [0]
```

Memory model (step 5)


global
city_to_temps
id1
temps
id4
id1
dict
'Toronto'
id7
'Ottawa'
id4
id2
int
3
id3
int
5
id4
list
0
1
2
id2
id3
id5
id5
int
8
id6
int
0
id7
list
0
id6

Audience Response

Interactive widget

Audience Response

Interactive widget

Audience Response

Interactive widget

Audience Response

Interactive widget

Audience Response

Interactive widget

A `for` loop over a `dict` goes through its **keys**. Use the key to look up its value.

```
total = 0
for item in item_to_price:
    total = total + item_to_price[item]
```

looping over a `dict`

```
counts = {}
for word in words:
    if word not in counts:
        counts[word] = 0
    counts[word] = counts[word] + 1
```

building a `dict`

To build a `dict`, start with `{}` and add a key the first time it's seen.

Python Workspace

Interactive widget

express_checkout.ipynb

The express checkout is for grocery orders with 8 or fewer items. Complete the examples in the docstring and then complete the function body.

In [ ]

```
# An order qualifies for express checkout if it does not have more items than
# the EXPRESS_LIMIT
EXPRESS_LIMIT = 8

def express_checkout(product_to_quantity: dict[str, int]) -> bool:
    """Return True if and only if the grocery order in product_to_quantity
    qualifies for the express checkout. product_to_quantity maps products
    to the numbers of those items in the grocery order.

    >>> express_checkout({'banana': 3, 'soy milk': 1, 'peanut butter': 1})
    True
    >>> express_checkout({'banana': 3, 'soy milk': 1, 'twinkie': 5})
    False
    """
    total = 0
    for product in product_to_quantity:
        total = total + product_to_quantity[product]
    return total <=EXPRESS_LIMIT
```

In [5]

```
express_checkout({'banana': 3, 'soy milk': 1, 'twinkie': 1})
```

```
True
```

build_placements.ipynb

As part of a study funded by a major shoe company, we crouched at the finish line of the Boston marathon and kept an ordered list of the shoes we saw as runners passed the finish line. Write a function that, given such a list, returns a dictionary mapping shoe companies to a list of the placements achieved by runners wearing shoes made by those companies.

In [ ]

```
def build_placements(shoes: list[str]) -> dict[str, list[int]]:
    """Return a dictionary where each key is a company and each value is a
    list of placements by people wearing shoes made by that company.

    >>> result = build_placements(['Saucony', 'Asics', 'Asics', 'NB', 'Saucony',
    ...                            'Nike', 'Asics', 'Adidas', 'Saucony', 'Asics'])
    >>> result == {'Saucony': [1, 5, 9], 'Asics': [2, 3, 7, 10], 'NB': [4],
    ...            'Nike': [6], 'Adidas': [8]}
    True
    """
```

In [ ]

```
build_placements(['Saucony', 'Asics', 'Asics', 'NB', 'Saucony', 'Nike', 'Asics', 'Adidas',
'Saucony', 'Asics'])
```

dicts and .csv files

When a `.csv` file has a header row, **`csv.DictReader`** gives each of the other rows as a `dict`. The keys are the column names from the header.

```
>>> import csv
>>> with open('watchlist.csv') as watch_file:
...     reader = csv.DictReader(watch_file)
...     for row in reader:
...         print(row['Title'], row['Rating'])
...
Whiplash 9
Shrek 8
Coraline 8
```

the first `row` is

`{'Title': 'Whiplash', 'Year': '2014', 'Genre': 'Drama', 'Rating': '9'}`

`row['Rating']` is easier to read than `row[3]`.
A constant such as `RATING = 'Rating'` makes it clearer still.

`data.csv` has one header row with column labels, then 20 rows of data. How many times is `print` called?

```
f = open('data.csv')
reader = csv.reader(f)
for row in reader:
    print(row)
f.close()
```

34 responses

`data.csv` has one header row with column labels, then 20 rows of data. How many times is `print` called?

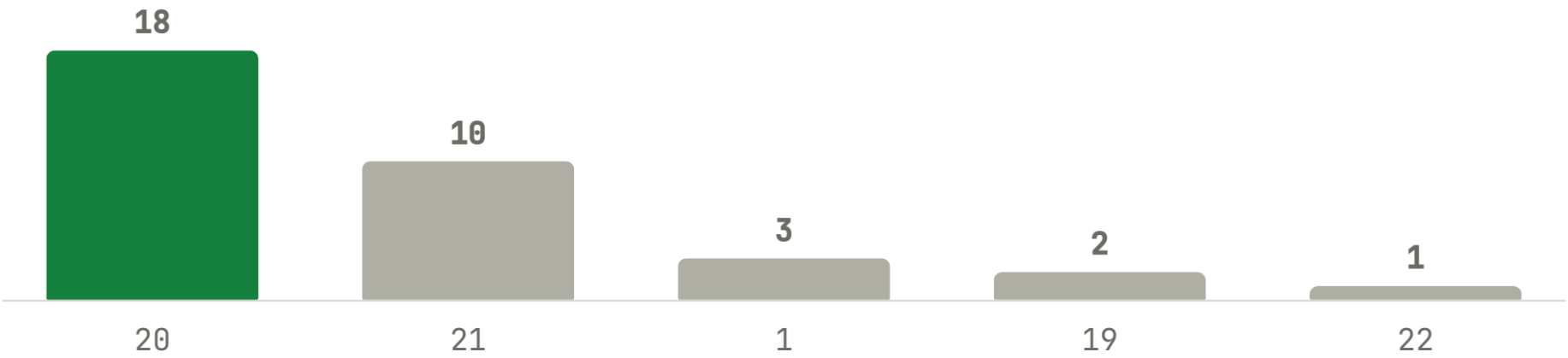
```
f = open('data.csv')
reader = csv.DictReader(f)
for row in reader:
    print(row)
f.close()
```

20
CORRECT ANSWER

53%
GOT IT RIGHT · 18 OF 34

20
MOST COMMON

5
DISTINCT ANSWERS



group_by_size.ipynb

Run this cell first: it creates `boba_orders.csv`.

In [ ]

```
with open('boba_orders.csv', 'w') as _f:
    _f.write('drink_name,size,price,is_dairy_free\nTaro Milk Tea,large,6.50,N\nMatcha
Latte,medium,5.75,N\nStrawberry Fruit Tea,large,6.00,Y\nBrown Sugar Milk
Tea,medium,5.75,N\nMango Fruit Tea,small,4.50,Y\nThai Milk Tea,large,6.50,N\n')
```

Complete the function according to its docstring.

In [ ]

```
import csv
from typing import TextIO

NAME = 'drink_name'
SIZE = 'size'
PRICE = 'price'
DAIRY_FREE = 'is_dairy_free'

def group_by_size(data_file: TextIO) -> dict[str, list[str]]:
    """Return a dictionary mapping each size to a list of drink names of that
    size, in the order they appear in the data_file.

    Assume the provided file is open for reading and has the same format as the
    boba_orders.csv example file. Your code should work for any such file.

    >>> f = open('boba_orders.csv')
    >>> result = group_by_size(f)
    >>> result == {'large': ['Taro Milk Tea', 'Strawberry Fruit Tea', 'Thai Milk Tea'],
    ...            'medium': ['Matcha Latte', 'Brown Sugar Milk Tea'],
    ...            'small': ['Mango Fruit Tea']}
    True
    >>> f.close()
    """
```

In [ ]

```
f = open('boba_orders.csv')
print(group_by_size(f))
f.close()
```